

IIC 2440 – Procesamiento de Datos Masivos

Tarea 1

Instrucciones. Esta tarea debe resolverse en grupos de a dos personas.

El formato de entrega consta de los siguientes archivos:

- Un archivo comprimido con todo el código usado para el trabajo (pero no los datos).
- Un informe (máximo 8 páginas) donde se reporten los resultados y las decisiones de diseño.

El uso de herramientas de IA generativa está permitida, y se recomienda su uso. Sin embargo, esta tarea promueve un uso responsable y ético, se asume que los alumnos son responsables por todo el material generado, y comprenden y son familiares con el trabajo realizado. El equipo docente se reserva el derecho de bajar la nota a aquellos alumnos o grupos que no lo demuestren.

Fechas. La fecha de entrega de la tarea es el **[POR DEFINIR]**, a las 20:00 hrs.

Introducción. En esta tarea vamos a diseñar e implementar un **data warehouse** sobre una colección masiva de artículos de noticias chilenas, y vamos a construir analítica distribuida usando **MapReduce**.

El proyecto simula un observatorio nacional de noticias, encargado de analizar tendencias, narrativas, diferencias regionales y dinámicas temporales en la cobertura mediática. No se requiere ningún motor de base de datos externo: el warehouse se implementará como **datasets estructurados almacenados como archivos** (simulando arquitecturas tipo Data LakeHouse en parquet).

Datos. Recibirán un archivo CSV (`noticias_chile_2023_2025.csv`) con noticias chilenas publicadas entre 2023 y 2025. Cada fila tiene la siguiente estructura:

`article_id, title, body, publish_date, source, country`

donde:

- **article_id:** identificador único del artículo (hash).
- **title:** título del artículo.
- **body:** cuerpo completo del artículo.
- **publish_date:** fecha de publicación (formato YYYY-MM-DD).
- **source:** dominio del medio de comunicación (ej. `emol.com`, `24horas.cl`).
- **country:** país (en este dataset, siempre **Chile**).

El dataset contiene varios millones de artículos provenientes de más de 100 medios distintos.

Estructura del trabajo. Este trabajo consta de dos partes. En la parte 1, tendrán que diseñar un ETL que trabaje con la información y transforme estos datos a un formato en particular, con una distribución de archivos que se asemeja a los sistemas basados en parquet, e incluyendo consultas que se aseguren de validar que la información fue procesada correctamente. En la segunda parte, usando funciones de Map Reduce, tendrán que procesar estos datos, calculando distintas analíticas que son importantes para el contexto de las noticias.

Parte 1: Data Warehouse (50 %)

1.1 Esquema Estrella.

Deben definir una **Fact Table** `fact_news` y las **tablas de dimensiones** correspondientes.

Tabla de hechos: La fact table debe tener al menos los hechos importantes de cada noticia (título, cuerpo, etc), pero también deberán calcular otros datos de interes (word count de body, titulo, etc.).

Tablas de dimensiones: Deberán calcular las siguientes dimensiones de los datos. Algunos son mas sencillos (como mes y año), pero otras como Región son más complejas.

- `dim_date` (fecha, mes, año, día de la semana, etc.)
- `dim_source` (medio de comunicación)
- `dim_region` (ver más abajo)

Su esquema debe asegurarse de que no haya duplicación en las dimensiones, y que las llaves sean las correctas.

1.2 Formato de Almacenamiento. El warehouse debe almacenarse como archivos (CSV, JSON, Parquet, etc.). La tabla de hechos debe estar particionada por **año y mes**, dejando todos los archivos de esa fecha en esa carpeta. Las dimensiones posibles se deben almacenar también en un archivo en la carpeta correspondiente también. Entonces, una estructura de archivos de acuerdo a este esquema:

```
warehouse/  
  dim_date/  
  dim_source/  
  dim_region/  
  fact_news/year=2023/month=05/  
  fact_news/year=2023/month=06/
```

1.3 Pipeline ETL. Deben implementar un pipeline ETL reproducible:

CSV crudo → Limpieza y normalización → Construcción de dimensiones → Asignación de llaves de dimensiones → Generación de tabla de hechos → Almacenamiento particionado

El pipeline completo debe poder ejecutarse con un solo script.

1.4 Enriquecimiento: extracción de región. Como parte del ETL, deben inferir la **región de Chile** a la que hace referencia cada artículo, usando **expresiones regulares** sobre el `title` y el `body`. Chile tiene 16 regiones administrativas:

- | | |
|-------------------------------|----------------|
| ■ Arica y Parinacota | ■ Maule |
| ■ Tarapacá | ■ Ñuble |
| ■ Antofagasta | ■ BíoBío |
| ■ Atacama | ■ La Araucanía |
| ■ Coquimbo | ■ Los Ríos |
| ■ Valparaíso | ■ Los Lagos |
| ■ Metropolitana (de Santiago) | ■ Aysén |
| ■ O'Higgins | ■ Magallanes |

Deben construir una tabla de alias (por ejemplo, "Santiago", "RM", "capital" → Metropolitana; "Concepción", "Biobío" → Bío-Bío; etc.) y aplicar regex insensibles a mayúsculas y acentos. Para resolver ambigüedades (un artículo puede mencionar varias regiones), deben **definir y justificar** una regla de desambiguación en el informe. Algunas opciones razonables:

- Priorizar coincidencias en el `title` por sobre el `body`.
- Voto por mayoría: asignar la región con mayor número de menciones.
- Combinar ambas (título como desempate del voto).

Los artículos sin ninguna mención detectable deben asignarse a una región especial **Desconocida** (o equivalente), de manera que *todos* los registros de `fact_news` tengan una región válida, para que quepan bien en el esquema estrella.

1.5 Validación Deben implementar al menos 5 validaciones que aseguren la consistencia del warehouse:

1. Consistencia referencial: no deben existir llaves foráneas (o llaves subrogadas de dimensiones) huérfanas.
2. El conteo de filas en la tabla de hechos debe coincidir con el número de artículos crudos.
3. No deben existir llaves duplicadas dentro de una misma tabla de dimensiones.
4. La distribución de particiones debe ser correcta (cada artículo en su partición correspondiente).
5. Una validación adicional que consideren relevante.

Parte 2: Análítica y MapReduce (50 %)

Todos los análisis de esta parte deben implementarse usando el paradigma MapReduce de forma manual. Es decir, deben programar explícitamente las funciones `map` y `reduce` en Python (pueden usar `functools.reduce`, iteradores, etc.), procesando los archivos directamente. **No se permite usar PySpark, Pandas, ni ninguna librería de procesamiento distribuido.** La idea es que entiendan la mecánica del modelo MapReduce sin usar librerías que provean de interfaces. **Nota importante:** Algunas de las tareas requieren más de una pasada de MapReduce (por ejemplo, calcular frecuencias regionales y globales por separado y luego combinarlas). Esto es esperable: pueden encadenar múltiples pasadas Map/Reduce. Además, cuando una pasada de MapReduce produce un resultado pequeño (por ejemplo, conteos diarios), el post-procesamiento sobre ese resultado intermedio puede hacerse con Python estándar sin necesidad de otra pasada MapReduce.

2.1 Top-K términos mensuales. Deberán calcular los 20 términos más frecuentes por año-mes.

Salida esperada (ejemplo):

2023-05, congreso, 12873

Recomendamos trabajar con un diccionario de stopwords para eliminar (es decir, no mapear) palabras comunes pero que no proveen información, como artículos, preposiciones, etc.

2.2 Distribución de palabras por región. Deberán calcular las frecuencias de palabras por región.

Para cada par (región, término), deben computar:

- Cantidad de ocurrencias regionales
- Cantidad de ocurrencias globales

- Razón de frecuencia relativa (regional /global)

Los artículos asignados a la región **Desconocida** pueden introducir ruido en este análisis; pueden optar por excluirllos, pero deberán justificar la decisión en el informe.

Para acelerar este cómputo y evitar problemas numéricos, considerar solo aquellas palabras que tienen una frecuencia global mayor o igual a un cierto número k . Si usaron un diccionario de stopwords a eliminar en la parte 2.1, recomendamos usarlo aquí también.

2.3 Divergencia de vocabulario por fuente. Para cada medio de comunicación, calculen la divergencia de su vocabulario respecto a la distribución global. Pueden usar comparación de frecuencias relativas o una métrica aproximada tipo KL.

2.4 Detección de peaks. Detectar días con aumentos anormales en el volumen de artículos.

1. Calcular el conteo diario de artículos.
2. Calcular un promedio móvil (por ejemplo, semana móvil).
3. Detectar peaks con respecto a ese promedio móvil.

2.5 Informe A partir de los resultados de la Parte 2, deberán escribir un breve análisis donde:

- Identifican tendencias interesantes en la cobertura mediática.
- Comparan el vocabulario entre regiones o fuentes.
- Analizan los peaks detectados y proponer posibles explicaciones.
- Presentan al menos una visualización que apoye sus conclusiones.